

单细胞高维向量近似最近邻检索系统

A Web-based ANN Retrieval Platform for Single-cell Data

软件工程大作业 · 课程答辩

FastAPI

React 19

FAISS · HNSWLIB

RAG

团队 XXX · 2026 年 5 月

目录 Outline

#	章节	关键词
1	项目背景	单细胞测序 · ANN 必要性
2	需求分析	课程要求 · 功能矩阵 · 加分项
3	系统设计	架构 · 技术栈 · 引擎抽象 · 数据模型
4	核心实现	检索流水线 · 关键代码
5	加分功能	多数据集联合 · Adaptive HNSW · RAG
6	实测与交付	性能数据 · 演示截图 · 质量保障 · 总结

1. 项目背景 Background

单细胞测序 (scRNA-seq) 数据特征

- 一次实验可生成 十万级 细胞样本，每个细胞数值化为高维向量
- 本项目使用 CZI 公开数据 `liver.h5ad`：69 032 细胞 × 30 维 PCA 向量

痛点

- 精确 KNN 在 $N=10^5$ 、 $D=30\sim 512$ 时延迟达数百毫秒，难以支持交互式查询
- 单细胞分析平台普遍依赖 Jupyter 脚本，缺乏面向 Web 的统一检索入口

ANN 的价值

- 在召回率仅损失 $< 0.5\%$ 的前提下，将单次查询从亚秒级压到 微秒级
- 配合元数据过滤、可视化、自然语言问答，可承载真实科研场景

2. 需求分析 Requirements

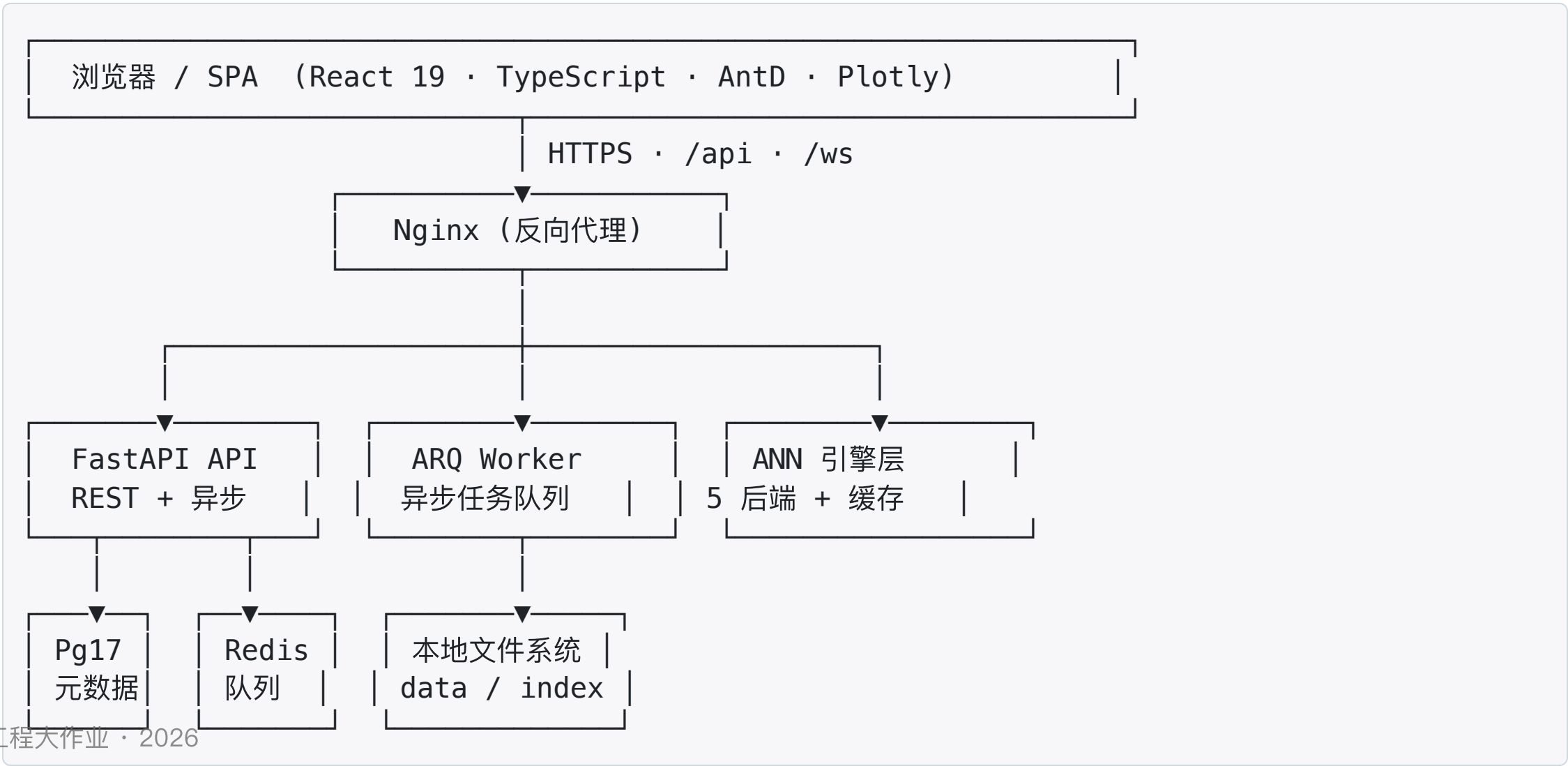
课程核心要求（必做）

模块	核心交付
用户信息	注册 / 登录 / JWT 鉴权 / 管理员角色
数据管理	.h5ad 上传、Scanpy 预处理、向量提取
索引构建	多后端 / 可保存可加载 / 异步任务
查询检索	按 cell_id / 按向量 / 条件过滤 / Top-K
可视化	UMAP 投影、检索结果高亮、性能图表

加分功能（全做）

- 多数据集联合检索：并发 + min-max 归一化 + 重排
- ANN 算法改进：自适应 HNSW（自研后端）
- RAG + 单细胞：自然语言 → 解析 → 检索 → 总结

3. 系统架构总览 Architecture



3.1 技术栈 Tech Stack

层	选型	用途
前端	React 19 · TypeScript · Vite · Ant Design · Zustand · Plotly	SPA · 状态管理 · 交互式可视化
后端	Python 3.12 · FastAPI · SQLAlchemy 2 async · Pydantic v2	异步 REST · ORM · 数据校验
任务队列	ARQ + Redis	索引构建 / 预处理后台任务
数据库	PostgreSQL 17	用户 / 数据集 / 索引 / 检索日志
缓存	Redis 7	查询缓存 + ARQ broker
ANN 引擎	FAISS · HNSWLIB · scikit-learn	5 种后端 (HNSW · IVF-PQ · Brute · Adaptive)
单细胞	scanpy · anndata · numpy · scipy	h5ad 读取 · PCA / UMAP
LLM	DashScope · OpenAI 兼容 · Mock	RAG 自然语言查询
基础设施	Docker Compose · Nginx · GitHub Actions · pre-Commit	部署 · CI · 代码规范

3.2 ANN 引擎抽象 IndexBackend

统一接口

```
IndexBackend (ABC)
├─ name           ── 后端标识
├─ build(X, **)   ── 构建索引
├─ search(q, k)   ── Top-K 检索
├─ save / load    ── 持久化
└─ memory_mb()    ── 内存估计
```

5 个具体后端 + 工厂 + 缓存

后端	实现库	特点
brute	numpy	精确，作为 ground truth
hnswlib	hnswlib 0.8.0	高召回 + 微秒级延迟
faiss-hnsw	faiss 1.13	OMP 多线程友好
faiss-ivfpq	faiss 1.13	量化压缩，内存极小
adaptive-hnsw	自研	早停 + 升档（加分项）

4. 核心代码：抽象 + 自适应 ef

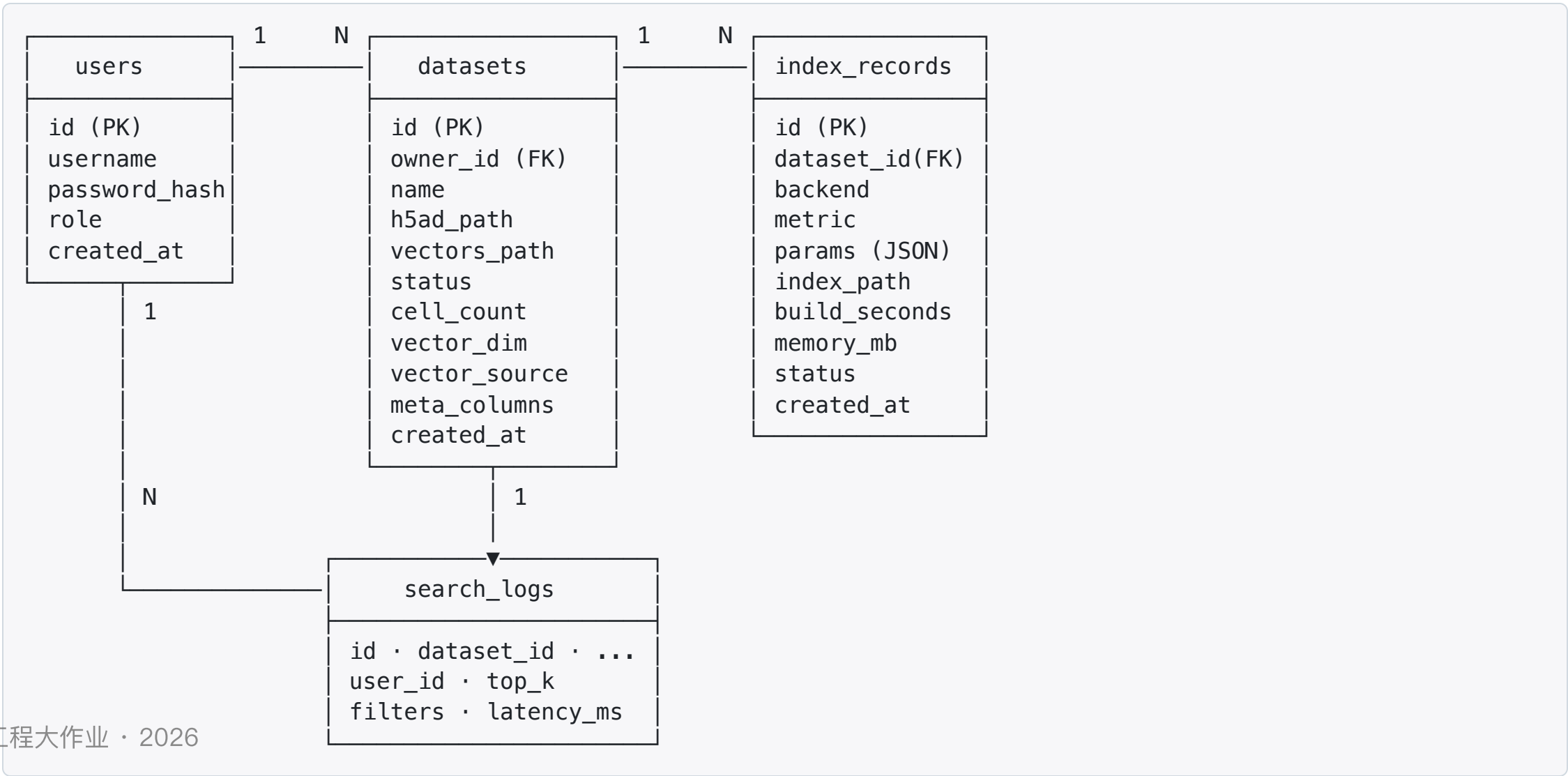
IndexBackend 抽象

```
class IndexBackend(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...
    @abstractmethod
    def build(self, vectors: np.ndarray, **params) -> None: ...
    @abstractmethod
    def search(self, query: np.ndarray, top_k: int) -> tuple[np.ndarray, np.ndarray]: ...
    @abstractmethod
    def save(self, path: str) -> None: ...
    @abstractmethod
    def load(self, path: str) -> None: ...
    @abstractmethod
    def memory_mb(self) -> float: ...
```

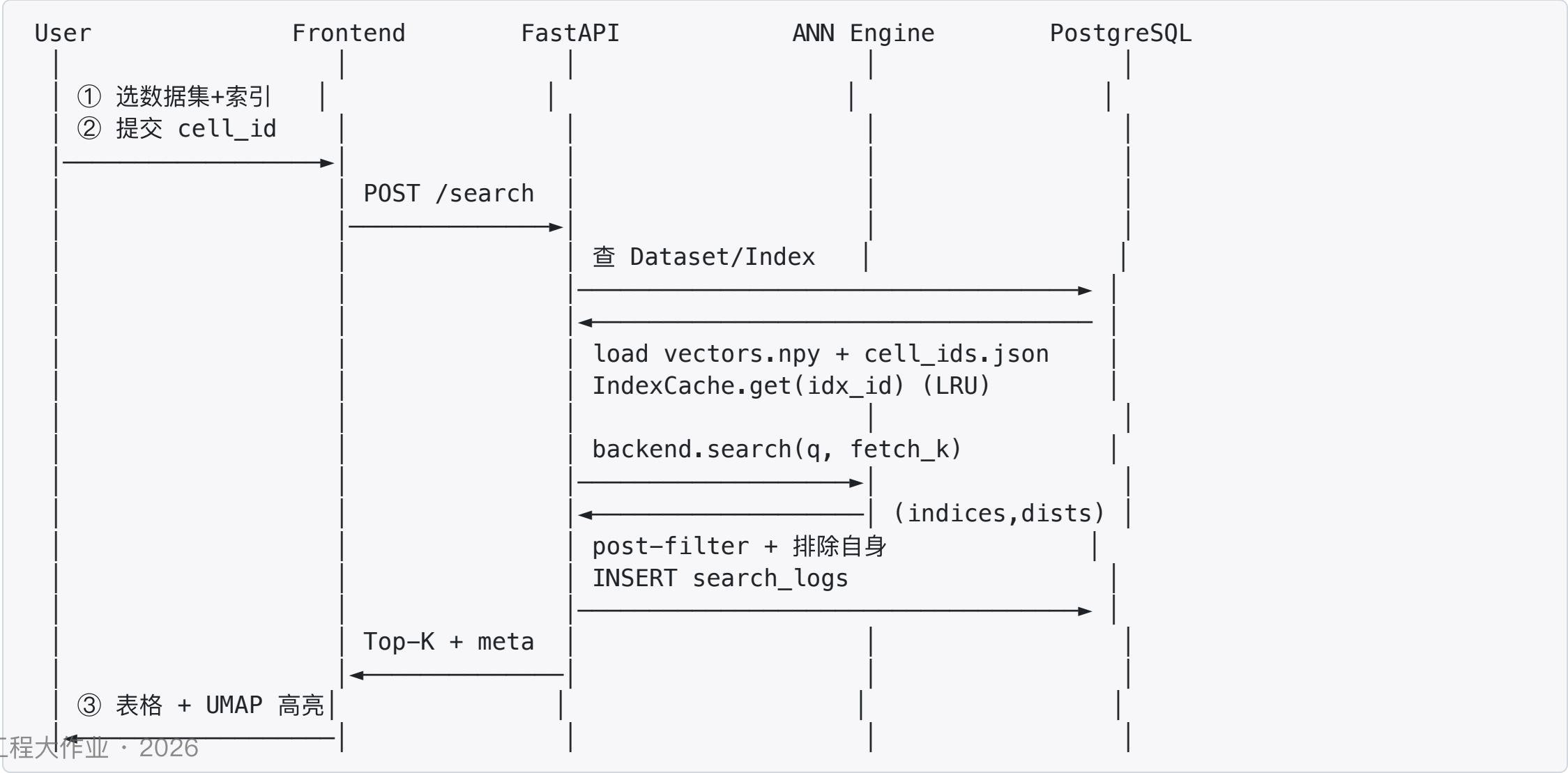
自适应 HNSW 核心循环（节选）

```
while pending.size > 0:
    self._index.set_ef(int(max(ef, k_query)))
    labels, dists = self._index.knn_query(q[pending], k=k_query)
    if retry == 0: # 首轮：相对距离间隔早停
```

4.1 数据模型 ER



4.2 检索流水线时序



5. 加分项 ①：多数据集联合检索

问题：用户希望"在多个肝脏数据集里一起找相似细胞"

实现要点

1. 路由层 `POST /search/multi-dataset`，接收 `dataset_ids: [int]`
2. 对每个数据集 **并发** 调用 `async_search_by_vector` (`asyncio.gather`)
3. 每个数据集结果做 **min-max 归一化**：`norm = (d - dmin) / (dmax - dmin)`
4. 合并后按 `normalized_distance` 升序，统一 Top-K，并标注 `source_dataset_id`

```
merged.sort(key=lambda x: x["normalized_distance"])
final = merged[:top_k]
for i, item in enumerate(final, start=1):
    item["rank"] = i
```

收益：跨数据集语义可比；并发 $\text{latency} \approx \max(\text{per_dataset})$ ，而非求和。

5. 加分项 ②：自适应 HNSW

动机：固定 `ef_search` 在 query 难度分布不均时浪费算力 / 召回不足。

策略（继承 `HnswlibBackend`，仅重写 `search`）

- 起始 `ef = min_ef (32)`，oversample 多取 8 个候选
- 首轮判稳：相对距离间隔 $gap = (d[k] - d[k-1]) / (d[k-1] - d[0])$ ，越大说明 Top-K 边界越清晰
- 后续轮判稳：与上一轮 Top-K 的集合重合度 `overlap@k`
- 升档：`ef = min(ef * 2, max_ef=512)`；按 query 粒度独立判定，已稳定提前返回

实测 (top_k=10, N=30 000)

指标	hnswlib 固定 ef=64	adaptive-hnsw
p50 / p95 (ms)	0.016 / 0.020	0.045 / 0.057
Recall@10	0.9996	0.9994
mean_ef	64	50.6

5. 加分项 ③：RAG 自然语言查询

流程： `parse_query` → ANN search → `summarize`



三客户端协议化设计 —— 无外网也可演示

6. 实测数据：liver.h5ad

数据集：CZI 儿童肝脏 scRNA-seq, 69 032 细胞 × 30 维 PCA (基准 N=30 000)

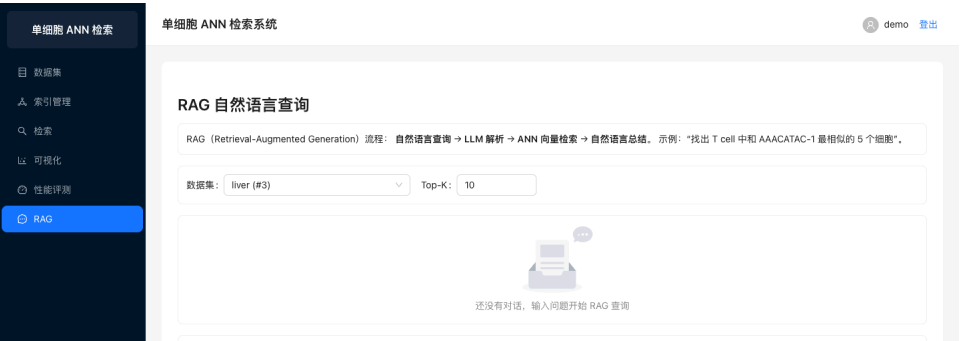
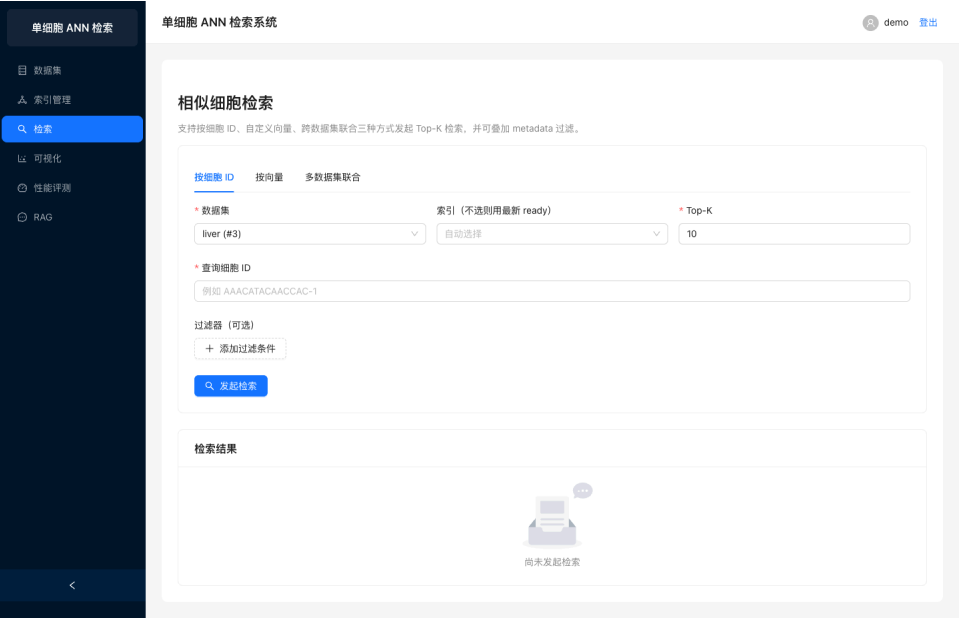
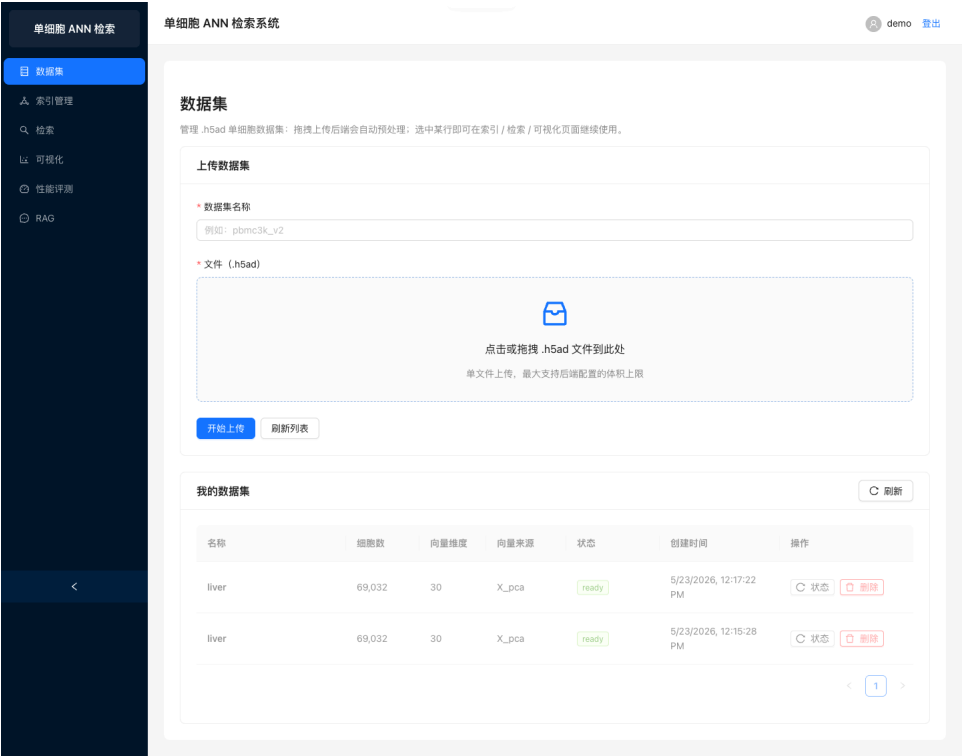
索引构建 (10 核 arm64, 单进程)

backend	构建 (s)	内存 (MB)	关键参数
brute	0.000	3.43	—
hnswlib	0.224	7.10	M=16, ef_c=200
faiss-hnsw	0.245	3.43	M=16, ef_c=200
faiss-ivfpq	0.187	0.29	nlist=173, m=10
adaptive-hnsw	0.218	7.10	M=16, ef_c=200

单线程 Top-10 检索延迟与召回

backend	p50 (ms)	p95 (ms)	p99 (ms)	QPS	Recall@10
brute	0.582	0.632	0.650	1 736	1.000
hnswlib	0.016	0.020	0.021	63 158	0.9996

6.1 演示截图集锦



6.2 质量保障 Quality

维度	指标
后端测试	35 个 pytest 用例 (auth · datasets · ann · search · evaluation · rag · health), 全部通过
静态检查	ruff lint + format · mypy 类型 · pre-commit 全绿
前端检查	eslint + prettier · TypeScript strict 模式
CI	GitHub Actions 三步流水线: lint → test → build
E2E	Playwright 跑通真实数据全链路, 自动产出 9 张验收截图
Git	25 次 约定式提交 (feat: / fix: / docs: / chore: / build: / test:)
文档	5 篇软件开发文档 (01_项目概述 ~ 05_用户手册) + 性能基准报告 + 答辩 PPT

关键提交节点:

- feat(backend): 自适应 HNSW 后端 + 基准测试脚本 (加分项)
- feat(backend): RAG 自然语言查询模块 (加分项)
- feat(backend): 检索与性能评测模块 (条件过滤 + 多数据集联合 + Recall/QPS/延迟)

6.3 项目交付清单

代码仓库 (GitHub)

- backend/ — FastAPI 后端 (22 个 Python 模块 · 35 测试)
- frontend/ — React 19 + TS 前端 (6 业务页面)
- infra/ — Docker Compose · Nginx · CI
- e2e/ — Playwright 端到端测试
- Makefile — 一键 make up / migrate / test / lint

软件开发文档 (docs/)

文件	内容
01_项目概述.md	背景 · 目标 · 计划
02_需求分析与系统设计.md	用例 · 架构 · 接口 · 数据库 · UI
03_系统测试.md	用例 · 性能测试
04_项目管理.md	分工 · 进度 · 工具

总结 Conclusion

一个完整的端到端 Web 系统

注册 → 上传 → 预处理 → 多后端索引 → 条件检索 → 可视化 → RAG 问答

三个加分项全部落地

多数据集联合 · 自适应 HNSW · RAG 自然语言

实测可量产的性能

微秒级延迟 · 99.96% Recall@10 · 6 万 QPS

谢谢聆听 · 欢迎提问

Q & A